Laboratory Assignment 1: Parallel Programming with MPI
A Distributed Data Structure

Alessandro Fazio
alessandro.fazio@estudiantat.upc.edu
Jose Ricardo Arias Perez
jose.ricardo.arias@estudiantat.upc.edu

Group: cpds11
Course 2026/27, Q1

Concurrency, Parallelism and Distributed Systems (CPDS)
FIB – Universitat Politècnica de Catalunya

October 14th, 2026

1 A distributed data structure (Section 1.1)

The 12×12 matrix is split into 4 strips of 3 consecutive rows, one per process. Each process stores its strip in rows `1..maxn/size` of `xlocal`. Rows `0` and `maxn/size+1` are ghost rows that hold copies of the neighbours' edge rows. Each message carries exactly one row: `maxn` elements of type `MPI_DOUBLE`. All three programs below perform the same exchange and print `No errors` when run with 4 processes.

Question 1: statements S1–S12

`par_data_struct.c` **(blocking** `MPI_Send/MPI_Recv`**)**

```
S1:  MPI_Comm_rank(MPI_COMM_WORLD, &rank );
S2:  MPI_Comm_size(MPI_COMM_WORLD, &size );
S3:  MPI_Recv(xlocal[0], maxn, MPI_DOUBLE, rank - 1, 0, MPI_COMM_WORLD,
   &status);
S4:  MPI_Send(xlocal[1], maxn, MPI_DOUBLE, rank - 1, 1, MPI_COMM_WORLD);
S5:  MPI_Recv(xlocal[maxn/size+1], maxn, MPI_DOUBLE, rank + 1, 1,
   MPI_COMM_WORLD, &status);
S6:  MPI_Reduce( &errcnt, &toterr, 1, MPI_INT, MPI_SUM, 0,
   MPI_COMM_WORLD );
```

A blocking call does not return until its buffer can safely be reused (send) or holds the received data (receive). Process 0 has no previous neighbour and the last process has no next neighbour, so every call is guarded by an `if` on the rank. These guards keep the boundary processes from addressing ranks that do not exist. Tag 0 marks rows travelling downwards and tag 1 marks rows travelling upwards. Finally, `MPI_Reduce` (S6) adds the local error counters into `toterr` on process 0, which prints the verdict.

`par_data_struct_nonblocking.c` **(**`MPI_Isend/MPI_Irecv`**)**

```
S7:  MPI_Isend( xlocal[maxn/size], maxn, MPI_DOUBLE, rank + 1, 0,
   MPI_COMM_WORLD, &r[nreq++] );
S8:  MPI_Irecv( xlocal[0], maxn, MPI_DOUBLE, rank - 1, 0, MPI_COMM_WORLD
   , &r[nreq++] );
S9:  MPI_Isend( xlocal[1], maxn, MPI_DOUBLE, rank - 1, 1, MPI_COMM_WORLD
   , &r[nreq++] );
S10: MPI_Irecv( xlocal[maxn/size+1], maxn, MPI_DOUBLE, rank + 1, 1,
   MPI_COMM_WORLD, &r[nreq++] );
S11: MPI_Reduce( &errcnt, &toterr, 1, MPI_INT, MPI_SUM, 0,
   MPI_COMM_WORLD );
```

`MPI_Isend` and `MPI_Irecv` only start a transfer and return immediately. Each call stores an `MPI_Request` handle in `r[nreq++]`, so `nreq` ends up counting the requests this process actually posted: 2 for the boundary processes and 4 for the inner ones. The program cannot read the ghost rows, or modify the rows being sent, until the transfers have completed. A single `MPI_Waitall(nreq, r, statuses)` waits for all of them before the validation loops. Because no call blocks, the order in which the four operations are posted cannot cause a deadlock.

`par_data_struct_sendreceive.c` **(**`MPI_Sendrecv` + `MPI_PROC_NULL`**)**

```
S12: MPI_Sendrecv( xlocal[1], maxn, MPI_DOUBLE, prev_nbr, 1, xlocal[maxn
   /size+1], maxn, MPI_DOUBLE, next_nbr, 1, MPI_COMM_WORLD, &status );
```

Each `MPI_Sendrecv` combines one send and one receive in a single call, and MPI performs both without deadlock regardless of the order in which processes execute them. The exchange is organised as two shifts along the linear array. The first call shifts the last local row down to `next_nbr`. S12 shifts the first local row up to `prev_nbr` and receives the next neighbour's first row into the bottom ghost row.

The boundary processes use `MPI_PROC_NULL` as their missing neighbour, so no `if` statements are needed. A send to `MPI_PROC_NULL` completes at once without transferring anything. A receive from it completes at once and leaves the buffer untouched, so those ghost rows keep their initial `-1`.

2 A simple Jacobi iterative method (Section 1.2)

The outputs of `Jacobi` and `Jacobi_nb` (4 processes) are identical to `Jacobi-out.txt`. The output of `Jacobi_vr` (5 processes, `maxn`=13) is identical to `Jacobi_vr_p5r13-out.txt`. We checked all three with `diff`, which reported no differences.

Question 1: Why `MPI_Allreduce` instead of `MPI_Reduce`?

Each process only computes the part of `diffnorm` that belongs to its own segment. The loop condition `gdiffnorm > 1.0e-2 && itcnt < 100` is evaluated by *every* process, so every process needs the global value. `MPI_Reduce` leaves the result only on the root (process 0), and the other processes would read an uninitialised `gdiffnorm`. They could then stop at a different iteration than process 0, or never stop, and the neighbour exchanges would deadlock or pair up wrongly. `MPI_Allreduce` combines the partial sums and delivers the result to all processes, so they all take the same decision in the same iteration.

Question 2: What would be needed with `MPI_Reduce`?

After the `MPI_Reduce` to process 0, an `MPI_Bcast` of `gdiffnorm` from process 0 to all processes would be required:

```
MPI_Reduce( &diffnorm, &gdiffnorm, 1, MPI_DOUBLE, MPI_SUM, 0,
   MPI_COMM_WORLD );
MPI_Bcast ( &gdiffnorm, 1, MPI_DOUBLE, 0, MPI_COMM_WORLD );
```

`MPI_Allreduce` does both steps in one call.

Question 3: Which code defines the halo?

The halo is defined by the loop bounds of the computation, which never write the boundary of the global mesh:

- **First and last columns:** the inner loop is `for (j=1; j<maxn-1; j++)`, so columns `0` and `maxn-1` are read (as `xlocal[i][j-1]` and `xlocal[i][j+1]`) but never written.
- **First and last rows:** the row range `i_first..i_last` is adjusted:

  ```
  i_first = 1;
  i_last  = maxn/size;          /* nrows in Jacobi_vr.c */
  if (rank == 0)         i_first++;
  if (rank == size - 1)  i_last--;
  ```

  Process 0 skips its local row 1 (global row 0), and the last process skips its last local row (global row `maxn-1`). These rows are set to `-1` by `xlocal[i_first-1][j] = -1;` `xlocal[i_last+1][j] = -1;` and are only read as neighbours.

- The copy-back loop `xlocal[i][j] = xnew[i][j]` uses the same bounds, so it does not overwrite the halo either.

For the inner processes, `xlocal[0]` and `xlocal[i_last+1]` are ghost rows. They are refreshed by communication in every iteration and are also only read. On the boundary processes, the corresponding communication uses `MPI_PROC_NULL` (S13), so the halo rows are never overwritten.

Question 4: Load balancing when `maxn` is not divisible by `P`

Each process calls `getRowCount(maxn, rank, size)`:

```
return (rowsTotal / mpiSize) + (rowsTotal % mpiSize > mpiRank);
```

Every process gets the base share of $\lfloor \texttt{maxn}/P \rfloor$ rows. The remainder $r = \texttt{maxn} \bmod P$ is always smaller than P. The comparison evaluates to 1 for the processes with rank $< r$ and to 0 for the rest, so the r leftover rows go one each to the first r processes. As a result, the row counts of any two processes differ by at most one, which is the best possible balance with whole rows.

For `maxn`=13 and P=5: $13/5 = 2$ and $13 \bmod 5 = 3$, so processes 0, 1 and 2 get 3 rows and processes 3 and 4 get 2 rows. The receive counts are `recvcnts` = {39, 39, 39, 26, 26} (`nrows*maxn`), and the displacements are their prefix sums, `displs` = {0, 39, 78, 117, 143}. `MPI_Gatherv` uses these to place each segment at the correct position in `x`.

Question 5: statements S13–S24

`Jacobi.c`

```
S13: if (next_nbr >= size) next_nbr = MPI_PROC_NULL;
S14: MPI_Allreduce( &diffnorm, &gdiffnorm, 1, MPI_DOUBLE, MPI_SUM,
                    MPI_COMM_WORLD );
S15: MPI_Gather( xlocal[1], maxn * (maxn/size), MPI_DOUBLE,
                 x, maxn * (maxn/size), MPI_DOUBLE,
                 0, MPI_COMM_WORLD );
```

This program reuses the exchange from Section 1.1 inside the Jacobi loop. `MPI_PROC_NULL` (S13) removes the boundary special cases without any branching and leaves the halo rows of the first and last process untouched. `MPI_Allreduce` (S14) provides the global convergence value to every process (see Question 1). After convergence, `MPI_Gather` (S15) collects the `maxn*(maxn/size)` values owned by each process. Each contribution starts at `xlocal[1]` and excludes the ghost rows. Process 0 receives the contributions in rank order, contiguously in `x`, and prints the full solution.

`Jacobi_nb.c`

```
S16: MPI_Wait( &r[2], &status );   /* fix: status passed by address */
S17: MPI_Waitall( nreq, r, statuses );   /* r[0] and r[1] */
S18: MPI_Wait( &r[3], &status );
S19: MPI_Iallreduce( &diffnorm, &gdiffnorm, 1, MPI_DOUBLE, MPI_SUM,
                     MPI_COMM_WORLD, &r[0] );
S20: MPI_Igather( xlocal[1], maxn * (maxn/size), MPI_DOUBLE,
                  x, maxn * (maxn/size), MPI_DOUBLE,
                  0, MPI_COMM_WORLD, &r[0] );
```

This version hides part of the communication latency. It posts the four non-blocking transfers and then computes rows `i_first..i_last-1`, which do not need the lower ghost row, while that row is still in transit. The program waits only where the data is actually needed:

- **S16** waits on `r[2]` so that the upper ghost row `xlocal[0]` has arrived before the first local row is computed. The original statement passed `status` instead of `&status`, which was the error.
- **S17** waits on `r[0]` (send of the last local row) and `r[1]` (receive of the lower ghost row) before computing the last row `i_last`. That row needs `xlocal[i_last+1]`, and the sent row must not change while it is in flight. Since `nreq`=2, `MPI_Waitall` completes exactly the first two entries of `r`.
- **S18** waits on `r[3]` so that the old `xlocal[1]` has been sent to the previous neighbour before it is overwritten with the new values.

S19 and S20 use the non-blocking collectives `MPI_Iallreduce` and `MPI_Igather`. They use `r[0]` because the code waits on `MPI_Wait(&r[0], &status)` right after each call, and `r[0]` is free since S17 has already completed it. Because the wait comes immediately after each call, with no independent work in between, these collectives give no real overlap here. They behave like their blocking versions and only illustrate the syntax.

`Jacobi_vr.c`

```
S21: return (rowsTotal / mpiSize) + (rowsTotal % mpiSize > mpiRank);
S22: nrows = getRowCount(maxn, rank, size);
S23: MPI_Gather( &lcnt, 1, MPI_INT, recvcnts, 1, MPI_INT, 0,
                 MPI_COMM_WORLD );
S24: MPI_Gatherv( xlocal[1], lcnt, MPI_DOUBLE,
                  x, recvcnts, displs, MPI_DOUBLE,
                  0, MPI_COMM_WORLD );
```

S21 and S22 distribute the rows as explained in Question 4. Each process now owns a different number of rows, so a plain `MPI_Gather`, which receives the same count from every process, cannot rebuild `x`. Collection therefore takes two steps:

1. **S23:** every process sends its element count `lcnt = maxn*nrows` to process 0, which fills `recvcnts` and computes `displs` as prefix sums.
2. **S24:** `MPI_Gatherv` sends `lcnt` values from each process, starting at `xlocal[1]`. Process 0 stores the i-th contribution of `recvcnts[i]` values at offset `displs[i]` in `x`.